까우룽까우룽(kauroong) 확정안 v2

Oct 23, 2026 · @동변동불주먹

1. 요약

팀 결정 9가지 중 6가지는 저장소 코드 수정이 필요하고, 가장 큰 변경은 구 경계 위치 검사, 제보 우선 + 예측 단계 계산, 1분 집계다. 까우룽까우룽(kauroong)은 하단 탭 4개(지도, 피드, 제보, 나무)로 이루어진 모바일 웹 서비스다. 사용자가 서울 25개 구에서 단풍 사진을 직접 촬영해 제보하면 AI 검증(CLOVA Studio)과 필요 시 운영자 검토를 거쳐 공개되고, 지도에는 구별 단계와 관심도 나무가 1분마다 갱신된다. 제보가 부족한 구는 산림청 2026 단풍절정 예측을 기본값으로 보여준다.

인프라(DB, 백엔드, 프런트)는 팀원이 구축을 마쳤다. 이 문서는 2026-10-23에 받은 저장소 백엔드 코드(`backend/app`)와 대조해 작성했고, 프런트 코드와 `cloud_function/` 폴더는 아직 보지 못했다.

2. 확정 결정과 코드 현황

코드 수정이 필요한 곳은 2, 3, 4, 5, 8, 9번이다. 1, 6, 7번은 이미 구현되어 있어 설정과 시드 값만 손보면 된다.

#	항목	팀 확정	현재 코드	필요한 변경
1	제보 상태	업로드 대기 → AI 검증 → 승인 / 검토 / 반려, 운영자 검토	구현됨. `PENDING_UPLOAD` → `PENDING_AI` → `APPROVED` / `REVIEW` / `REJECTED`, `UPLOAD_FAILED`. 검증 작업은 임대 토큰으로 중복 실행을 막는다	코드 변경 없음. 운영 환경에서 `VERIFICATION_MODE=clova`, `CLOVA_API_KEY` 설정
2	사진 입력	직접 촬영	서버는 파일 업로드를 받고 촬영 여부를 구분하지 못한다	프런트를 카메라 전용으로 변경 (개발 모드에서만 갤러리·샘플 허용)
3	위치 검사	구 경계 안	구역 중심점과 반경(`radius_m`)의 거리 비교(`distance_m`)	25개 구 경계로 판정하고, 선택한 구와 다르면 422
4	단계 결정	제보 우선, 부족하면 예측값	최근 7일 승인 제보 중 가장 최근 1건의 단계(`refresh_zone`), 없으면 `UNKNOWN`	최근 3일 승인 제보 3건 이상이면 최신 10건 다수결, 아니면 예측 기반

#	항목	팀 확정	현재 코드	필요한 변경
5	갱신 주기	1분	승인·좋아요 때마다 즉시 재집계하고, 워커가 60초마다 전체 구역을 재집계(`reconcile`)	즉시 재집계와 구역 락 제거, 집계 엔드포인트 신설, 1분 크론이 호출
6	나무·좋아요	관심도 = 제보×2 + 제보자×3 + 좋아요(7일), 레벨 0~3	구현됨. `ZoneActivity` 가 같은 공식으로 점수를 내고 40점 이상 3, 15점 이상 2, 1점 이상 1	계산식은 유지하고 계산 시점만 1분 집계로 옮긴다
7	포인트	승인 시 20P, 하루 한도 없음, 아이템 가격은 우리 값	구현됨. `report_points=20` , 한도 없음, 적립 키 `report:{id}:approved`	아이템 시드 값 교체(12절 질문)
8	DB	우리 설계 기반, MySQL 통일	SQLAlchemy 모델 + Alembic, 기본은 SQLite, 운영은 MySQL 필수	6절 참고
9	Cloud Functions	1분 주기	업로드 이벤트를 처리하는 함수(`handle_storage_event`)만 있다	1분 크론 함수 추가, 함수 두 개 운영

추가 결정:

- AI 검증 워커와 운영자 검토 코드는 유지하고 활성화한다(CLOVA Studio).
- 관심도 레벨 0~3은 그대로 쓰고, 지도 나무 수는 레벨과 같다.
- 로컬 개발을 포함해 DB는 MySQL로 통일한다.
- 저장소는 public으로 그대로 쓰고 나중에 삭제한다. 데이터 출처 문구는 삭제 전까지 유지한다.
- 서비스 영문 표기는 `kauroong` 이다.

3. 규칙 상세

승인된 제보만 공개·집계·포인트 대상이 되고, 구별 단계는 제보가 있으면 제보로, 부족하면 산림청 예측으로 정한다.

제보 처리

1. 제보 탭에서 위치 권한을 허용하고 웹 카메라로 촬영한 뒤 단계(`GREEN` , `STARTING` , `PEAK` , `FALLEN`)와 수종(`MAPLE` , `GINKGO` , `MIXED`)을 고른다. 요청에는 `Idempotency-Key` 가 있다.
2. API가 사진 형식·용량·디코딩, **좌표가 25개 구 경계 안인지와 선택한 구와 일치하는지(불일치 422)**, 제보 간격(429)을 검사한다.
3. DB에 제보와 검증 작업을 먼저 만들고 사진을 Object Storage에 올린다. 응답은 202이고, 사진 EXIF·GPS는 제거한다.
4. 업로드 이벤트로 검증 함수가 CLOVA Studio에 판별을 요청한다. 결과는 승인 / 검토 / 반려이고, 오류가 3번 나면 검토로 보낸다.
5. 검토 제보는 운영자가 승인하거나 반려한다. 반려 사유는 본인의 제보 화면에 보인다.

제보 상태는 `PENDING_UPLOAD` → `PENDING_AI` → `APPROVED` / `REVIEW` / `REJECTED` 이고, 사진이 끝내 올라오지 않으면 `UPLOAD_FAILED` 다.

단계 결정 (1분마다 구별 계산)

1. **제보 기반**: 구의 최근 3일 승인 제보가 3건 이상이면 최신 10건의 다수결로 정한다. 동률이면 동률 단계 중 가장 최근 제보의 단계다. 출처 표시는 `REPORT` .
2. **예측 기반**: 위 조건이 아니면 오늘 날짜와 구의 예상 절정일로 계산한다. 출처 표시는 `FORECAST` .
3. 예측도 없으면 `UNKNOWN` 으로 두고 지도에서 회색으로 표시한다. 기획서의 "정보 부족" 표시는 폐지한다.

d = 오늘 − 예상 절정일	단계
d ⩽ −14	`GREEN` (물들기 전)
−14 < d < −3	`STARTING` (물드는 중)
−3 ⩽ d ⩽ 3	`PEAK` (절정)
d > 3	`FALLEN` (지는 중)

구의 예상 절정일은 구 안 예측 지점의 평균이고, 지점이 없으면 서울 기본값 10/31이다. 지점별 날짜는 산림청 2026 수종별 예측(전국)으로 은행나무류 10/30, 단풍나무류 10/31이다. 구역 시트에는 "제보 기반 / 예측 기반"을 표시한다.

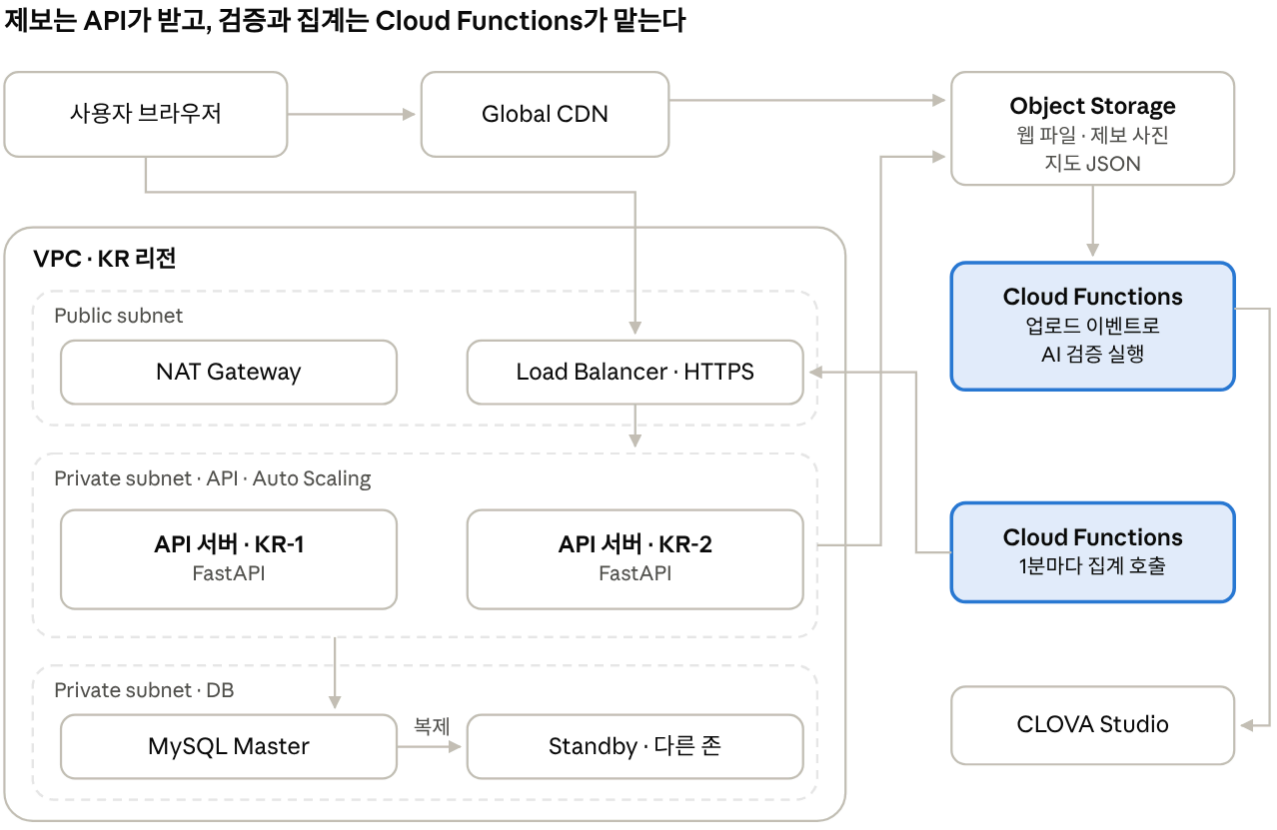
관심도와 나무

- 대상은 최근 7일에 생성된 승인 제보이고, 좋아요 수는 그 제보들에 달린 좋아요의 합이다.
- `점수 = 제보 수 × 2 + 서로 다른 제보자 수 × 3 + 좋아요 수`
- 레벨은 0(점수 0), 1(1~14), 2(15~39), 3(40 이상)이고, 지도의 나무 수는 레벨과 같다(0~3그루).
- 좋아요는 단계나 사진의 진위를 결정하지 않고 관심도에만 반영된다.

포인트와 아이템

- 제보가 처음 승인될 때 20P를 1회 적립한다. 적립 키 `report:{id}:approved`가 이중 적립을 막고, 하루 한도는 없다.
- 좋아요는 포인트와 연결하지 않는다.
- 아이템은 3개이고 중복 구매할 수 없다. 포인트가 모자라면 구매할 수 없다(409). 장착은 본인이 가진 아이템만 가능하다.
- 아이템 가격은 20P, 30P, 50P다. 이름과 슬롯은 12절의 질문으로 정한다.
- 제보 간격 제한은 현재 코드에서 사용자당 5초(`REPORT_COOLDOWN_SECONDS`)다. 운영 값은 12절에서 정한다.

4. 시스템 구조



강조한 박스는 Cloud Functions 두 개이고, Read Replica와 Redis는 이번 범위에서 제외했다.

시스템 구조 · VPC 안 API와 MySQL, 밖에 Cloud Functions 두 개

사용자는 CDN과 로드밸런서로 들어오고, 제보 사진은 API 서버가 Object Storage에 올린다. 업로드 이벤트 함수가 CLOVA Studio로 사진을 검증하고, 1분 크론 함수가 집계 API를 호출한다. 코드상 검증 함수는 DB에 직접 접속하므로 함수에서 DB로 가는 경로는 12절에서 확인한다.

5. 요청 흐름



제보가 지도에 반영되는 흐름 · 접수, AI 검증, 승인, 1분 집계

제보는 접수 직후 202로 응답하고 검증은 비동기로 진행된다. 승인된 제보만 포인트와 지도에 반영되고, 지도에는 다음 1분 집계 때 나타난다. 검토 제보는 운영자가 승인해야 같은 길로 들어간다.

6. DB: 저장소 스키마에 얹는 변경

저장소의 SQLAlchemy 스키마(문자열 ID, Alembic)를 유지하고, 우리 설계 요소를 마이그레이션 하나로 얹는 방식을 권장한다. "우리 설계 기반"을 우리가 만든 BIGINT 기반 ERD로 전체를 교체하라는 뜻으로 정했다면 모델, API, 프런트가 모두 바뀌므로 알려 달라(12절).

이 방식에서는 기존 SQL v1 파일(BIGINT, 우리 테이블 이름)은 직접 실행하지 않고 구 경계·예측 지점 데이터의 출처로만 쓴다.

테이블	변경	이유
`zones`	`boundary` (구 경계) 추가, `latitude` · `longitude` 는 대표점으로 갱신, `radius_m` · `revision` 제거. 5개 샘플 구역을 25개 구로 교체	구 경계 안 판정, 구역 락 제거

테이블	변경	이유
`zone_status`	`source` (`REPORT` / `FORECAST` / `NONE`)와 `peak_date` 추가. `stage` 계산 규칙 변경	제보 기반 / 예측 기반 표시
`forecast_spots`	신규. `id`, `name`, `kind` (`MOUNTAIN` / `GINKGO`), `latitude`, `longitude`, `zone_id`, `peak_date`, `source`	예측 지점 25곳과 구 매핑
`zone_activity`	변경 없음	관심도 점수와 `tree_level`이 이미 결정과 같다
`reports`, `report_likes`, `report_verifications`, `verification_jobs`	변경 없음	AI 검증 흐름이 이미 구현되어 있다
`users`, `point_ledger`	변경 없음	`email`, `password_hash`, `role`, `last_report_at` (제보 간격), 적립 키가 이미 있다
`items`, `user_items`, `tree_equipment`	시드 값만 교체	아이템 이름·가격·슬롯
`seasons`	변경 없음	`id`가 연도(예: `2026`)다

새 변경은 Alembic 리비전 하나로 만들고, 현재 head는 `947f5a336531`이다. 구 25개와 예측 지점 25곳은 같은 마이그레이션의 시드로 넣는다.

코드에서 같이 바뀌는 곳

- `main.py`의 `create_report`: 반경 비교를 구 경계 판정으로 교체하고 `lock_zone` 호출을 제거한다.
- `services.py`: `distance_m`을 구 판정으로 대체하고, `decide_report`와 `set_like`의 즉시 재집계·구역 락을 제거하고, `refresh_zone`을 제보 + 예측 계산으로 교체한다.
- `jobs.py`: `reconcile`에서 구역 재집계를 빼고 업로드 복구만 남긴다.
- `main.py`: 집계 엔드포인트(비밀 키로 보호, 동시 실행 방지)와 지도 JSON 업로드를 추가하고, `/api/zones`에 `source`, `peak_date`를 포함한다.
- `cli.py`: 샘플 구역 5개와 샘플 제보 2건을 25개 구 기준으로 바꾼다.

7. 데이터 자산과 출처

구 경계 25개와 예측 지점 25곳이 준비되어 있고, 저장소의 문자열 구역 ID에 맞추는 일만 남았다.

자산	파일	상태
서울 25개 구 경계 + 대표점	`04_zones_seoul25.sql`	25개 구, 모두 유효한 단일 Polygon, 대표점은 구 안에 포함
예측 지점 25곳	`forecast_spots.csv`, `05b_forecast_spots_seed.sql`	구마다 1곳, 구 매핑 검증 통과
단계 계산 로직	`foliage_logic.py`	자체 테스트 통과. 승인 제보 기준과 저장소 단계 이름에 맞춰 이식 예정
로컬 DB 구성	`setup_local_db.ps1`	Windows + Docker에서 실행 확인(구 SQL v1 기준)

구역 ID 제안

저장소의 `zones.id` 는 문자열이고 지금은 영문 슬러그다(예: `seoulforest`). 구 경계 SQL의 번호(행정구역 코드순)를 영문 슬러그로 바꿔 쓰는 안을 제안한다.

No	ID	구	No	ID	구
1	`jongno`	종로구	14	`mapo`	마포구
2	`jung`	중구	15	`yangcheon`	양천구
3	`yongsan`	용산구	16	`gangseo`	강서구
4	`seongdong`	성동구	17	`guro`	구로구
5	`gwangjin`	광진구	18	`geumcheon`	금천구
6	`dongdaemun`	동대문구	19	`yeongdeungpo`	영등포구
7	`jungnang`	중랑구	20	`dongjak`	동작구
8	`seongbuk`	성북구	21	`gwanak`	관악구
9	`gangbuk`	강북구	22	`seocho`	서초구
10	`dobong`	도봉구	23	`gangnam`	강남구
11	`nowon`	노원구	24	`songpa`	송파구
12	`eunpyeong`	은평구	25	`gangdong`	강동구
13	`seodaemun`	서대문구			

프런트가 구역 ID를 문자열 그대로 쓰는지 확인한 뒤 확정한다.

출처

- 구 경계: 통계청 SGIS 행정동 경계(공공누리 제1유형)를 가공한 파생물, CC BY 4.0. `04_zones_seoul25.sql` 맨 위의 출처 문구는 저장소를 삭제하기 전까지 유지한다.
- 예상 절정일: 산림청 '2026년 한반도 단풍절정 예측지도'(2026-09-22 발표)의 수종별 값을 서울 전 지점에 적용했다. 서울 개별 지점의 산림청 예측일은 확인하지 못했다. 발표 자료에는 "수종별 전국 예측 기준"이라고 표기한다.
- 지점 좌표는 대부분 Google Places에서 확인했다. public 저장소에 올리되 삭제 예정이다.

8. 기획서(10/16) 대비 달라진 점

오제보 대응, 공개 시점, 갱신 주기, 단계·나무 규칙 네 곳이 크게 바뀌었다.

항목	기획서	확정안
오제보 대응	이미지 판별은 다음 단계, 운영자 검토 없음	AI 검증(CLOVA Studio)과 운영자 검토를 이번 범위에 포함
제보 공개 시점	제출하면 피드에 바로 공개	승인된 제보만 공개
제보 상태	대기 · 통과 · 숨김	업로드 대기 · AI 검증 대기 · 승인 · 검토 · 반려 · 업로드 실패
위치 검사	구역 반경 안	구 경계 안
갱신 주기	1시간	1분
구역 단계	최근 제보 다수결, 부족하면 "정보 부족"	제보 우선, 부족하면 산림청 예측 기반
나무 수	제보 수 비율, 최대 N그루	관심도 레벨 0~3그루
좋아요	포인트·나무와 무관	관심도에 반영(포인트와는 무관)
포인트	수치 미정	승인 시 20P, 하루 한도 없음
구역	시·구 단위	서울 25개 구 경계 폴리곤

미룰 것은 벚꽃나무 테마, 나무 성장 단계, 신고 기능, 댓글이다.

9. 남은 작업

작성 지원이 맡는 일은 1~5번과 11번이고, 인프라·프런트와 설정은 팀이 맡는다. 1번부터 순서대로 진행하면 의존이 없다.

순서	작업	수정 위치	담당
1	구 25개 경계와 ID를 모델·마이그레이션·시드에 반영(`zones.boundary` , `zone_status` · `zone_activity` 초기 행, `forecast_spots` 25곳)	`models.py` , Alembic 새 리비전, `cli.py`	작성 지원(초안) + 팀(적용)
2	좌표 → 구 판정, 선택한 구와 다르면 422	`services.py` , `main.py` 의 `create_report` , 의존성 추가	작성 지원
3	단계 계산(제보 다수결 + 예측)과 구별 집계 함수	`services.py` 의 `refresh_zone` 교체	작성 지원
4	즉시 재집계와 구역 락 제거	`services.py` , `main.py` , `jobs.py` 의 `reconcile`	작성 지원 + 팀
5	집계 엔드포인트(비밀 키, 동시 실행 방지)와 지도 JSON 업로드	`main.py` , `storage.py`	작성 지원
6	Cloud Functions 1분 크론 함수 배포, 지도 JSON CDN 캐시 60초 이하	인프라	팀
7	CLOVA 활성화 (`VERIFICATION_MODE=clova` , `CLOVA_API_KEY`), 업로드 이벤트 함수 점검과 함수의 DB 접속 확인	환경변수, `jobs.handle_storage_event`	팀
8	아이템 시드 교체	`cli.py` 의 `ITEMS`	팀(12절 확인 후)
9	프런트: 카메라 전용, 구 폴리곤 색칠, 레벨 나무, "제보 기반 / 예측 기반" 표시, 지도 JSON 읽기	`frontend/`	팀
10	이름 통일: 토큰의 `iss` · `aud` 값이 `kkawooroong` 으로 되어 있다	`security.py` , 프런트, 문서	팀
11	시연 데이터(구마다 다른 단계·관심도)와 부하 테스트 스크립트	`scripts/`	작성 지원
12	기획서와 README 갱신	문서	팀 + 작성 지원

10. 인프라 확인 체크리스트

팀원이 DB, 백엔드, 프런트 인프라 구축을 마쳤으므로, 새 마이그레이션을 적용하기 전에 아래를 확인한다.

- [ ] Cloud DB for MySQL 버전이 8.0 이상인지, `kauroong` DB와 접속 계정·권한이 있는지
- [ ] DB 접속 경로(API 서버나 점프 서버에서 실행하는지)와 ACG(DB는 API 서버의 3306만 허용)
- [ ] 백엔드 환경변수: `ENVIRONMENT=production`이면 `JWT_SECRET`은 32자 이상, `DATABASE_URL`은 MySQL, `STORAGE_BACKEND=s3`, `VERIFICATION_MODE=clova`, `PUBLIC_API_URL`은 HTTPS여야 한다
- [ ] Object Storage 버킷: 웹 파일, 제보 사진(비공개 ACL), 지도 JSON, 그리고 `S3_ENDPOINT_URL`·`S3_BUCKET`·키 설정
- [ ] 업로드 이벤트 함수가 DB에 직접 접속해야 한다(`handle_storage_event`가 DB 세션을 연다). 함수에서 private 서브넷의 DB에 닿는지
- [ ] 1분 크론 함수 배포와 집계 엔드포인트 비밀 키 보관
- [ ] CLOVA Studio 키는 코드에 넣지 않고 환경변수나 Secret으로 관리
- [ ] HTTPS(Certificate Manager)가 Load Balancer에 연결되었는지(카메라·위치는 HTTPS에서만 동작)
- [ ] 지도 JSON의 CDN 캐시 60초 이하, Auto Scaling 최소 서버 수와 시연 전 증설 계획
- [ ] Naver Maps Client ID에 서비스 도메인이 허용되어 있는지

11. 시연 참고

시연과 부하 테스트는 `ENVIRONMENT=test`, `VERIFICATION_MODE=demo` 환경에서 하는 것이 안전하다. 운영 설정(`production`)에서는 테스트 판별 결과(`demo_outcome`)를 쓸 수 없고 제보마다 실제 CLOVA 호출이 일어나 비용과 시간이 든다.

- 구마다 예측일이 10/30(은행길 17곳) 또는 10/31(산·공원 8곳)뿐이라, 예측만으로는 10/28~11/2에 지도 전체가 같은 단계로 보인다. 구별로 다른 색을 보여주려면 샘플 제보를 구마다 다른 단계로 섞어 넣거나, 예측 계산 날짜를 10/27이나 11/3으로 지정한다.
- 관심도 레벨 예시: 제보 1건(1명) = 5점 → 레벨 1, 제보 3건(3명) = 15점 → 레벨 2, 제보 8건(8명) = 40점 → 레벨 3이다. 시연 데이터를 이 점수에 맞춰 만든다.
- 부하 테스트는 지도 JSON과 피드 조회 위주로 하고, 제보 제출은 `demo` 모드 환경에서만 돌린다.
- 제보는 승인된 뒤에 공개되므로, 시연 전에 승인 흐름을 한 번 돌려 지연이 길어 보이지 않게 한다.
- 제보 간격 제한이 5초라서 한 계정으로 연속 제보하면 429가 나온다. 부하 테스트는 계정을 나눠서 한다.

12. 미확정·확인할 것

코드를 읽고 나서 정리된 항목은 포인트 20P, 하루 한도 없음, 좋아요 7일 범위(7일 안에 만든 제보에 달린 좋아요), 장착 슬롯 이름이다. 아래 7가지는 팀의 답이 필요하다.

#	항목	현재 상태	제안
1	DB 방침	"우리 설계 기반"의 뜻이 정해지지 않았다	저장소 스키마에 우리 요소를 얹는 방식을 권장한다. BIGINT 기반 ERD로 전체를 교체하는 안이면 모델·API·프런트가 모두 바뀐다
2	구역 ID	저장소는 문자열 ID(예: `seoulforest`)	7절의 영문 슬러그로 통일. 프런트가 ID를 문자열로 쓰는지 확인
3	제보 간격	코드는 사용자당 5초	시연은 5초 유지, 운영 값은 팀이 정한다(환경변수로 바꿀 수 있다)
4	GPS 정확도 기준	코드에 없다(좌표만 받는다)	이번 범위에서는 적용하지 않는다
5	아이템 시드	코드: 가을 리본 30P(`decoration`), 반짝이는 별 50P(`decoration`), 작은 버섯 20P(`ground`). 우리 안: 빨간 리본 20P, 전구 줄 30P, 노을 하늘 50P	가격 집합(20/30/50P)은 이미 같다. 이름까지 바꿀지, 슬롯(`decoration`, `ground` 두 개)을 그대로 쓸지 정한다
6	검증 함수의 DB 접속	코드상 업로드 이벤트 함수가 DB에 직접 접속한다	함수가 private 서브넷의 DB에 닿는 구성인지 확인
7	미열람 코드	프런트, `cloud_function/`, Alembic 파일	받으면 필드 단위로 다시 대조